Prompt de consulta utilizado 1: Necesito organizar mi avance individual del TPI de Metodología de Sistemas II sobre el caso de solicitudes académicas para asegurar que cubro los requisitos de la rúbrica

Para organizar tu avance individual y asegurarte de cubrir el 100% de la rúbrica de evaluación de Metodología de Sistemas II, conviene estructurar el trabajo en un plan de acción por etapas vinculadas directamente con los criterios de ponderación y los entregables obligatorios.

1. Diagnóstico de Calidad Interna y Externa (Ponderación: 15%)

El objetivo de este bloque es dejar en claro que un software puede satisfacer al usuario y, al mismo tiempo, tener una estructura interna deficiente:

- **Calidad externa:** Describir el alcance funcional del caso (registro de trámites de estudiantes como Federico Savastano, Juan Martinez, Marisa Amicuzi o Sonia Gonzalez, asignación de responsables docentes, cambios de estado y emisión de reportes). Demostrar que el sistema resuelve las operaciones sin fallos visibles.

- **Calidad interna:** Documentar la rigidez de la arquitectura base, la falta de cohesión, el acoplamiento excesivo y la dificultad para incorporar nuevas reglas sin modificar código crítico.

- **Evidencias clave:** Captura de la consola ejecutando la versión inicial y diagrama o esquema simple del diseño monolítico.

2. Deuda Técnica y Code Smells (Ponderación: 20%)

La consigna exige relevar al menos 8 problemas técnicos o debilidades de diseño:

- **Identificación exhaustiva:** Listar los 8 smells (God Class, Long Method, Long Parameter List, Primitive Obsession, condicionales anidados complejos, Feature Envy, código duplicado y atributos públicos sin encapsulamiento).

- **Estructura requerida por smell:** Ubicación exacta en el código, fragmento/evidencia, impacto en la mantenibilidad, riesgo técnico ante cambios futuros y nivel de prioridad de corrección.

- **Hito de repositorio:** Registrar el tag `etapa-diagnostico` tras documentar estos puntos.

3. Aplicación de SOLID y Clean Code (Ponderación: 20%)

En esta fase se justifica y aplica el rediseño teórico sobre el código:

- **Principios SOLID:**

    - **SRP:** Separar la clase central en un servicio de coordinación, un repositorio de persistencia en memoria y un reporteador de consola.

    - **OCP:** Encapsular la lógica de tiempos de resolución (SLA) para permitir nuevos tipos de solicitud sin alterar la lógica de negocio.
    - **DIP:** Hacer que el servicio dependa de una abstracción (`ISolicitudRepository`) y no de una implementación rígida.

- **Clean Code:** Eliminar abreviaturas crípticas en nombres de variables, asegurar métodos breves de propósito único y estandarizar excepciones de dominio para el control de errores.

4. Refactoring Seguro (Ponderación: 20%)

La cátedra evalúa transformaciones concretas que mejoren el diseño sin romper el comportamiento existente:

- **Técnicas mínimas requeridas:**
    1. *Introduce Parameter Object:* Agrupar los parámetros sueltos de creación en un objeto de solicitud.
    2. *Move Method:* Trasladar la validación y actualización de estados dentro de la entidad `Solicitud`.
    3. *Extract Method / Extract Class:* Modularizar la impresión y cálculo de métricas en un componente de reportería.
    4. *Replace Type Code with Enum:* Reemplazar cadenas de texto por enumerados tipados para estados, prioridades y tipos de trámite.
- **Evidencia antes/después:** Contrastar fragmentos de código donde se aprecie la reducción de complejidad ciclomática.
- **Hito de repositorio:** Registrar el tag `etapa-refactoring`.

5. Pruebas y Verificación del Comportamiento (Ponderación: 10%)

Garantiza que el refactoring no introdujo regresiones funcionales:

- **Prueba de regresión visual:** Ejecutar el flujo del sistema refactorizado con los mismos datos y verificar que la salida por consola coincida exactamente con la inicial.
- **Suite de verificación:** Implementar comprobaciones automáticas sobre reglas críticas (asignación de tiempos SLA, transiciones válidas de estado y bloqueo de modificaciones sobre solicitudes cerradas).
- **Hito de repositorio:** Registrar el tag `etapa-pruebas`.

6. Code Review Profesional (Ponderación: 10%)

Simula una revisión técnica de pares formal:

- **Checklist técnico:** Diseñar una tabla de control con aspectos de diseño, legibilidad, manejo de errores y encapsulamiento.
- **Registro de observaciones:** Anotar el estado de cada ítem, el feedback técnico y la acción correctiva aplicada en el código.
- **Ubicación:** Guardar el documento en la carpeta `docs/` del repositorio.

7. Repositorio, Documentación y Presentación (Ponderación: 5%)

Verificación de los aspectos formales y trazabilidad:

- **Tags obligatorios en GitHub:** Asegurarse de tener pusheados todos los tags: `estado-inicial`, `etapa-diagnostico`, `etapa-refactoring`, `etapa-pruebas`, `etapa-docs` y `estadorefactorizado`.
- **README.md:** Descripción del caso, alcance, instrucciones de ejecución y tabla de tags.
- **Justificación de modalidad:** Aclarar en la portada del PDF que la entrega es individual debido a la baja del compañero de equipo a mitad de cursada.
- **Declaración de uso de IA:** Especificar las consultas puntuales de apoyo técnico realizadas, indicando herramienta, objetivo y evidencias adjuntas en el anexo.

Prompt de consulta utilizado 2: Cómo conviene desacoplar el cálculo de horas SLA de un bloque if-else anidado en Java aplicando principios SOLID y refactorings seguros sin cambiar la salida por consola?

Para desacoplar el cálculo de horas SLA eliminando la complejidad ciclomática de los condicionales anidados sin alterar el comportamiento observable del sistema, la técnica más limpia y segura en Java consiste en combinar **Replace Type Code with Enum** con **Move Method / Strategy encapsulada en el Enum**.

Esta refactorización traslada la regla de negocio desde la capa de orquestación (`SistemaSolicitudes`) directamente al modelo de dominio, resolviendo la violación de los principios **Single Responsibility Principle (SRP)** y **Open/Closed Principle (OCP)**.

1. Diagnóstico del código base (Antes)

En la versión original, el método que procesa la solicitud asume múltiples responsabilidades y recurre a cadenas de texto desprotegidas (*Primitive Obsession*) y condicionales anidados:

Java

```
// Smell: Conditional Complexity, Primitive Obsession y violación de OCP
int horas = 0;
if (tipo.equals("EQUIVALENCIA")) {
    if (prio.equals("ALTA")) {
        horas = 24;
    } else if (prio.equals("MEDIA")) {
        horas = 72;
    } else {
        horas = 120;
    }
} else if (tipo.equals("CERTIFICADO")) {
    if (prio.equals("ALTA")) {
        horas = 12;
    } else {
        horas = 48;
    }
} else if (tipo.equals("MESA_EXAMEN")) {
    horas = 36;
} else {
    horas = 96;
}
```

Problemas técnicos asociados:

- **Violación de SRP:** El servicio de registro decide los tiempos de SLA en lugar de delegarlo en la entidad o valor de dominio correspondiente.
- **Violación de OCP:** Cada vez que el área académica define un nuevo trámite o modifica una ventana horaria, es necesario abrir y modificar la lógica de control principal.
- **Fragilidad:** Una errata en un literal `String` (ej. `"alta"` en minúsculas) desvía el flujo al bloque por defecto sin advertencia de compilación.

2. Solución desacoplada con Enums y Polimorfismo / Parámetros de Dominio

La solución recomendada consta de dos enums fuertemente tipados: `Prioridad` y `TipoSolicitud`. El propio enum `TipoSolicitud` conoce su matriz de tiempos o su algoritmo de cálculo mediante un método de dominio.

Paso A: Definir el enum de Prioridad

Java

```java
public enum Prioridad {
    ALTA,
    MEDIA,
    BAJA
}
```

Paso B: Encapsular los valores de SLA dentro de `TipoSolicitud`

```java
public enum TipoSolicitud {
    EQUIVALENCIA(24, 72, 120),
    CERTIFICADO(12, 48, 48),
    MESA_EXAMEN(36, 36, 36),
    RECLAMO_NOTA(96, 96, 96);

    private final int horasAlta;
    private final int horasMedia;
    private final int horasBaja;

    TipoSolicitud(int horasAlta, int horasMedia, int horasBaja) {
        this.horasAlta = horasAlta;
        this.horasMedia = horasMedia;
        this.horasBaja = horasBaja;
    }

    public int calcularHorasEstimadas(Prioridad prioridad) {
        if (prioridad == Prioridad.ALTA) {
            return this.horasAlta;
        }
        if (prioridad == Prioridad.MEDIA) {
            return this.horasMedia;
```

```
        }
        return this.horasBaja;
    }
}
```

Paso C: Delegación directa en la entidad `Solicitud`

Al instanciar la `Solicitud` mediante el parámetro agrupado (`NuevaSolicitudRequest`), el cálculo se vuelve declarativo y de una sola línea:

Java

```
public class Solicitud {
    // ...
    public Solicitud(int id, NuevaSolicitudRequest request) {
        this.id = id;
        this.tipo = request.getTipo();
        this.prioridad = request.getPrioridad();

        // Delegación de comportamiento: cálculo desacoplado y sin condicionales en el servicio
        this.horasEstimadas = this.tipo.calcularHorasEstimadas(this.prioridad);
        // ...
    }
}
```

3. Justificación bajo Principios de Diseño y Refactoring Seguro

1. **Open/Closed Principle (OCP):** Para incorporar un nuevo trámite (por ejemplo, `PRORROGA_CURSADA(48, 72, 96)`), solo se agrega una constante al enum `TipoSolicitud`. Ninguna clase existente de servicio, persistencia o controlador requiere modificación.

2. **Single Responsibility Principle (SRP):** El servicio `SistemaSolicitudes` se limita a coordinar el almacenamiento y la interacción con repositorios, desentendiéndose de la aritmética interna de plazos académicos.

3. **Preservación del comportamiento externo:** La tabla de mapeo mantiene con exactitud matemática los mismos valores originales:

    - `EQUIVALENCIA` + `ALTA` = **24h**, `MEDIA` = **72h**, `BAJA` = **120h**.

- `CERTIFICADO` + `ALTA` = **12h**, otras = **48h**.
- `MESA_EXAMEN` = **36h** uniformes.
- `RECLAMO_NOTA` (o casos residuales equivalentes) = **96h** uniformes.

4. **Verificación sin regresión:** Al ejecutarse el formateador de reportes (`reporteador.imprimirReporteGeneral(...)`), las cadenas de texto emitidas hacia la consola continúan imprimiendo exactamente `(SLA: 24h)`, `(SLA: 48h)`, etc., garantizando que los contratos de interfaz externa permanezcan idénticos.

¿Te gustaría agregar algún caso de borde adicional en la suite de pruebas para documentar la validación de estos tiempos de respuesta en el informe técnico?